REST ASSURED + CUCUMBER · SESIÓN 4 DE 4

Reportes, pipeline y examen final

GitHub Actions paso a paso: cada Pull Request se prueba solo, el resumen aparece en el PR y el reporte queda publicado en GitHub Pages. Después, el examen sobre el Portal Canvia Market.

Instructor **Giancarlo Palomino** Viernes 30 de octubre · 15:00 a 17:00 Examen: **feature/examen-<tu-nombre>**

HOY · 120 MINUTOS

Que la máquina pruebe por ti, y demostrar lo aprendido

HORA	MIN	QUÉ HACEMOS
15:00 – 15:40	40	GitHub Actions paso a paso: disparadores, caché de Maven, `mvn test`, reporte como artefacto, resumen de pruebas en el PR y publicación en GitHub Pages
15:40 – 15:50	10	`@smoke` en cada PR y regresión completa programada
15:50 – 16:50	60	**Examen final** sobre la API del Portal Canvia Market
16:50 – 17:00	10	Cierre y siguientes pasos

PARTIMOS DE

El proyecto con los ejercicios 1 a 5, en verde con `mvn test`, y el `BUGS.md` de ayer.

AL FINAL DEL DÍA

Tu repositorio prueba cada Pull Request sin que nadie lo pida y publica un reporte que cualquiera puede abrir con un enlace.

ANTES DE EMPEZAR

En tu repositorio: **Settings → Pages → Source: GitHub Actions**. Sin eso, el último paso del pipeline no puede publicar.

INTEGRACIÓN CONTINUA

Cada cambio, probado antes de entrar a develop

1 · TÚ
git push
abres o actualizas un Pull Request

→

2 · GITHUB
Actions arranca
una máquina Ubuntu limpia

→

3 · PIPELINE
mvn test
con Java 17 y la caché de Maven

→

4 · EVIDENCIA
Resumen y reportes
en el PR, como artefacto y en Pages

→

5 · DECISIÓN
Verde o rojo
se integra o se corrige

"En mi PC funciona"

La CI corre en una máquina que no es la tuya: si pasa ahí, no depende de tu configuración, tu caché ni tus archivos sin subir.

Nadie se olvida

Las pruebas no dependen de que alguien se acuerde de correrlas. Se ejecutan en cada push, en cada PR y cada mañana.

Un solo archivo

Todo vive en `.github/workflows/ci.yml`, versionado con el código. Lo revisas en un PR como cualquier otro cambio.

WORKFLOW → JOBS → STEPS

Cuatro palabras para leer cualquier pipeline

PALABRA	QUÉ ES	EN NUESTRO CI.YML
`on`	Cuándo corre	push, PR, programado y a mano
`jobs`	Trabajos; cada uno en una máquina nueva	`pruebas` y `publicar-reporte`
`steps`	Pasos de un job, en orden	9 pasos numerados en comentarios
`uses`	Usa una acción ya hecha	`actions/setup-java@v5`
`run`	Ejecuta un comando de consola	`mvn -B test …`
`needs`	Espera a otro job	`publicar-reporte` espera a `pruebas`

WORKFLOW · CI

.github/workflows/ci.yml

on: push · pull_request · schedule · workflow_dispatch

JOB 1 · PRUEBAS

checkout → Java → etiquetas → mvn test → resumen → artefacto → sitio

siempre

JOB 2 · PUBLICAR-REPORTE

deploy-pages

solo en develop, nunca en un PR

YAML: LA SANGRÍA ES SINTAXIS

Dos espacios por nivel, nunca tabulaciones. Un espacio de más mueve una clave a otro nivel y el workflow ni arranca.

PASO 1 · DISPARADORES

Cuándo corre: cuatro eventos, cuatro motivos

.github/workflows/ci.yml — YAML

```yaml
name: CI

# 1 · Cuándo corre
on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main, develop]
  schedule:
    - cron: '0 11 * * 1-5'          # lunes a viernes, 06:00 en Lima (11:00 UTC)
  workflow_dispatch:                # botón "Run workflow" en la pestaña Actions
    inputs:
      etiquetas:
        description: 'Filtro de etiquetas de Cucumber'
        required: true
        default: 'not @bug'
```

EVENTO	PARA QUÉ
`pull_request`	Probar tu cambio **antes** de integrarlo
`push`	Confirmar que `develop` sigue en verde después del merge
`schedule`	Detectar que la **API** cambió aunque nadie tocó el código
`workflow_dispatch`	Correr a mano, con las etiquetas que quieras

EL CRON VA EN UTC

`0 11 * * 1-5` = minuto 0, hora 11 UTC, de lunes a viernes. Lima está en UTC–5: son las 06:00. Corre sobre la rama por defecto (`develop`).

PASOS 2 Y 3 · CÓDIGO, JAVA Y CACHÉ

Una máquina limpia que no descarga Internet cada vez

.github/workflows/ci.yml · job pruebas — YAML

```yaml
    steps:
      # 2 · Traer el código
      - uses: actions/checkout@v6

      # 3 · Java 17 con caché de Maven (~/.m2): la segunda vez no descarga nada
      - uses: actions/setup-java@v5
        with:
          distribution: temurin
          java-version: '17'
          cache: maven
```

TEMURIN 17

La misma distribución y versión de Java que instalaste el primer día. Igual en tu PC y en la CI.

PRIMERA EJECUCIÓN

Descarga todas las dependencias

REST Assured, Cucumber, JUnit, Jackson… y guarda ~/.m2

SIGUIENTES EJECUCIONES

Restaura ~/.m2 desde la caché

mientras el pom.xml no cambie

CAMBIAS EL POM.XML

Caché nueva

la clave incluye un hash de los pom.xml

POR QUÉ IMPORTA

Menos minutos de espera en cada PR y menos dependencia de que Maven Central responda justo en ese momento.

PASOS 4 Y 5 · QUÉ CORRE Y EJECUTAR

El mismo mvn test de siempre, con otras etiquetas

.github/workflows/ci.yml · job pruebas — YAML

```yaml
      # 4 · Qué escenarios corren según el evento
      - name: Elegir etiquetas
        id: filtro
        env:
          EVENTO: ${{ github.event_name }}
          MANUAL: ${{ inputs.etiquetas }}
        run: |
          case "$EVENTO" in
            pull_request)      ETIQUETAS="@smoke and not @bug" ;;   # rápido:
feedback en minutos
            workflow_dispatch) ETIQUETAS="$MANUAL" ;;
            *)                 ETIQUETAS="not @bug" ;;             # push y
schedule: regresión completa
          esac
          echo "etiquetas=$ETIQUETAS" >> "$GITHUB_OUTPUT"
          echo "### Etiquetas: \`$ETIQUETAS\`" >> "$GITHUB_STEP_SUMMARY"

      # 5 · Ejecutar
      - name: Ejecutar escenarios
        env:
          ETIQUETAS: ${{ steps.filtro.outputs.etiquetas }}
        run: mvn -B test "-Dcucumber.filter.tags=$ETIQUETAS"
```

$GITHUB_OUTPUT

Guarda un valor para los pasos siguientes: `steps.filtro.outputs.etiquetas`.

$GITHUB_STEP_SUMMARY

Escribe Markdown en la página de resumen de la ejecución: se ve qué etiquetas se usaron.

¿POR QUÉ ENV Y NO ${{ }} DENTRO DE RUN?

Lo que escribe una persona (el filtro manual) no debe pegarse dentro de un comando: podría inyectar otro comando. Como variable de entorno, es solo texto.

-B

Modo *batch*: Maven no pregunta nada y el log sale sin colores ni barras de progreso.

PASOS 6 Y 7 · RESUMEN Y ARTEFACTO

El resultado, donde lo va a buscar quien revisa

.github/workflows/ci.yml · job pruebas — YAML

```yaml
      # 6 · Resumen de pruebas en la pestaña del job y como comentario en
el PR (aunque haya fallos)
      - name: Resumen de pruebas
        if: ${{ !cancelled() }}
        uses: mikepenz/action-junit-report@v6
        with:
          report_paths: target/cucumber-junit.xml
          check_name: Resultados de Cucumber
          include_passed: true
          detailed_summary: true
          comment: ${{ github.event_name == 'pull_request' }}

      # 7 · Reportes como artefacto descargable (aunque haya fallos)
      - name: Guardar reportes
        if: ${{ !cancelled() }}
        uses: actions/upload-artifact@v5
        with:
          name: reportes-cucumber
          path: |
            target/cucumber-report.html
            target/cucumber.json
            target/cucumber-junit.xml
            target/surefire-reports/
          retention-days: 14
```

IF: !CANCELLED()

Por defecto, un paso no corre si el anterior falló. Pero el reporte importa **justamente** cuando algo falla: estos pasos corren siempre, salvo que se cancele la ejecución.

`Resultados de Cucumber`

Resumen en el PR

La acción lee `cucumber-junit.xml` y crea un *check* con cada escenario, un comentario en el PR y el resumen en la página de la ejecución.

`reportes-cucumber.zip`

Artefacto

Los tres reportes, descargables desde la ejecución durante 14 días. Es la evidencia que se adjunta a un bug.

PASOS 8 Y 9 · GITHUB PAGES

El reporte con un enlace, sin descargar nada

.github/workflows/ci.yml · paso 8 — YAML

```yaml
      # 8 · Preparar el sitio de GitHub Pages (solo en develop, la rama por defecto; nunca en un PR)
      - name: Preparar sitio del reporte
        if: ${{ !cancelled() && github.event_name != 'pull_request' && github.ref == 'refs/heads/develop' }}
        run: |
          mkdir -p sitio
          cp target/cucumber-report.html sitio/index.html
      - name: Subir sitio para Pages
        if: ${{ !cancelled() && github.event_name != 'pull_request' && github.ref == 'refs/heads/develop' }}
        uses: actions/upload-pages-artifact@v4
        with:
          path: sitio
```

SOLO UNA VEZ POR REPOSITORIO

Settings → Pages → Source: GitHub Actions. El entorno `github-pages` solo acepta la rama por defecto: por eso la condición pide `develop`.

¿POR QUÉ NO EN UN PR?

El sitio muestra el estado de `develop`. La evidencia de un PR es el artefacto y el comentario.

.github/workflows/ci.yml · paso 9 — YAML

```yaml
  # 9 · Publicar el reporte en https://<usuario>.github.io/<repo>/
  publicar-reporte:
    name: Publicar reporte en GitHub Pages
    needs: pruebas
    if: ${{ !cancelled() && github.event_name != 'pull_request' && github.ref == 'refs/heads/develop' }}
    runs-on: ubuntu-latest
    permissions:
      pages: write                  # publicar en Pages
      id-token: write               # identificar el despliegue
    environment:
      name: github-pages
      url: ${{ steps.despliegue.outputs.page_url }}
    steps:
      - name: Desplegar
        id: despliegue
        uses: actions/deploy-pages@v4
```

PERMISOS, CONCURRENCIA Y SECRETOS

Lo mínimo necesario, y nada de claves en el YAML

.github/workflows/ci.yml — YAML

```yaml
# Permisos mínimos por defecto; cada job pide lo que necesita.
permissions:
  contents: read

# Si llega otro push a la misma rama, cancela la ejecución anterior.
concurrency:
  group: ci-${{ github.ref }}
  cancel-in-progress: true

jobs:
  pruebas:
    name: Pruebas de API
    runs-on: ubuntu-latest
    permissions:
      contents: read
      checks: write                 # resumen de pruebas como "check"
      pull-requests: write          # comentario con el resumen en el PR
```

PERMISSIONS

El token automático de la ejecución solo puede leer el código. El job de pruebas pide además escribir *checks* y comentarios; el de Pages, publicar.

CONCURRENCY

Si subes tres commits seguidos, solo importa el último: los anteriores se cancelan y no gastan minutos.

CREDENCIALES DE UNA API REAL

Se guardan en **Settings → Secrets and variables → Actions** como `API_USUARIO` y `API_CLAVE`, y se pasan al paso como variables de entorno con `secrets.API_CLAVE`. `Configuracion` las lee primero. GitHub las oculta en el log.

HOY NO HACE FALTA

Restful Booker usa las credenciales públicas del sandbox que trae el `pom.xml`.

@SMOKE EN CADA PR, REGRESIÓN PROGRAMADA

Rápido donde se espera, completo donde importa

EVENTO	ETIQUETAS	ESCENARIOS	PREGUNTA QUE RESPONDE
`pull_request`	`@smoke and not @bug`	3	¿Rompí lo esencial?
`push` a develop	`not @bug`	23	¿develop sigue sano después del merge?
`schedule` 06:00	`not @bug`	23	¿La API cambió durante la noche?
`workflow_dispatch`	la que escribas	—	¿Ya corrigieron los `@bug`?

POR QUÉ NO TODO EN CADA PR

Hoy son 23 escenarios en medio minuto. En un proyecto real son cientos y tardan. El PR necesita una respuesta en minutos; la regresión puede esperar a la noche.

EL RIESGO

Si algo que no es `@smoke` se rompe en un PR, se descubre en el push o a la mañana siguiente. Por eso `@smoke` se elige con cuidado: lo que no puede fallar.

PREGUNTA PARA LA SALA

De tus escenarios, ¿cuáles marcarías `@smoke`? ¿Uno de seguridad debería estar?

LO HACEMOS JUNTOS

Guion de la demo: del push al enlace del reporte

1. **Activar Pages** en Settings y mostrar la pestaña **Actions** vacía.
2. **Rama y PR** con el `ci.yml`: se dispara *Pruebas de API* solo con `@smoke`.
3. **Mientras corre:** abrir el log en vivo, ver la restauración de la caché y el filtro de etiquetas.
4. **Al terminar:** el check *Resultados de Cucumber*, el comentario en el PR y el artefacto.
5. **Merge a develop:** corre la regresión completa y aparece el job *Publicar reporte* con el enlace.
6. **Run workflow** a mano con `@bug`: rojo a propósito, y el resumen muestra los 8 bugs.

```
PS> git switch develop; git pull
PS> git switch -c feature/s4-pipeline
PS> git add .github/workflows/ci.yml
PS> git commit -m "ci: agrega pipeline con reporte en GitHub
Pages"
PS> git push -u origin feature/s4-pipeline
PS> gh pr create --base develop --fill
# paso 6, desde la consola:
PS> gh workflow run ci.yml --ref develop -f etiquetas=@bug
```

QUÉ MIRAR

El tiempo de **setup-java** en la primera ejecución y en la segunda: esa diferencia es la caché.

LEER EL ROJO DE LA CI

Errores típicos del pipeline y cómo salir de ellos

`Invalid workflow file`

YAML mal sangrado

Una tabulación o un espacio de más. GitHub marca la línea en la pestaña Actions. Revisa con la extensión de YAML de VS Code.

`Get Pages site failed`

Pages no está activo

Falta **Settings → Pages → Source: GitHub Actions**. Las pruebas pasan, pero el despliegue falla.

`not allowed to deploy`

Rama no permitida

El entorno `github-pages` solo acepta la rama por defecto. Publica desde `develop`, no desde tu rama.

`Resource not accessible by integration`

Faltan permisos

El job no pidió `checks: write` o `pull-requests: write`. Agrégalos en el job, no al workflow entero.

`Unknown lifecycle phase "and"`

Comillas perdidas

El `-Dcucumber.filter.tags=…` con espacios va entre comillas también en el YAML.

`verde en tu PC, rojo en la CI`

Algo que no subiste

Un archivo sin `git add` (un esquema JSON, un feature) o una dependencia instalada a mano. La CI solo ve lo que está en el repo.

EX

Examen final · Portal Canvia Market

Una API nueva, con autenticación, productos y pedidos. Todo lo de la semana, en una hora, con el pipeline en verde al final.

Autenticación
Token con `POST /api/auth`, enviado como Bearer.

CRUD
Productos y pedidos: crear, consultar, cambiar y borrar.

Validaciones
Código, cuerpo y esquema de cada respuesta.

Bugs
La API tiene bugs sembrados. Encuéntralos y documéntalos.

REGLAS Y TIEMPOS

Sesenta minutos, una rama y un Pull Request

HORA	QUÉ HACES
15:50 – 15:55	Lees el enunciado en el portal (`/examen`) y la documentación de la API (`/docs`)
15:55 – 16:35	Escenarios, clientes y validaciones. **Commit cada vez que algo pasa a verde.**
16:35 – 16:45	Pipeline en verde y `BUGS.md`
16:45 – 16:50	Pull Request a `develop` . A las 16:50 se evalúa lo que esté subido

LA API

Portal Canvia Market: lo levanta el instructor, también corre en tu PC (`node server.js` , puerto 3000) y en la CI. Apunta tu proyecto con `"-DbaseUrl=http://localhost:3000"` o la URL que se publique.

SE PERMITE

Tu propio código de la semana, las soluciones que hayas escrito, la documentación de REST Assured, Cucumber, JSON Schema y GitHub Actions, y los PDF del curso.

NO SE PERMITE

Copiar código de un compañero o compartir el tuyo durante el examen. Escenarios que dependen de ids fijos o del orden de ejecución.

CÓMO EMPEZAR

`git switch develop; git pull; git switch -c feature/examen-<tu-nombre>` . Un `.feature` por recurso, un cliente por recurso, igual que esta semana.

SI ALGO NO FUNCIONA

Primero lee el rojo (escenario, paso, aserción o excepción, log). Si a los 5 minutos sigues igual, deja el escenario con `@wip` y avanza.

RÚBRICA DEL SÍLABO

Cómo se califica · aprobación con 70 %

CRITERIO	PESO	QUÉ SE MIRA
Escenarios en Gherkin claros, en español y declarativos	20 %	Sin URLs ni JSON en el `.feature`; cada escenario crea sus datos
Autenticación, CRUD y casos negativos funcionando	30 %	Token Bearer, alta, consulta, cambio y baja; 400, 401 y 404 donde corresponde
Validaciones correctas (código, cuerpo y esquema)	20 %	No solo el código: contenido y JSON Schema
Pipeline en verde con el reporte publicado	20 %	CI en verde en el PR y reporte como artefacto o en Pages
Bugs encontrados y documentados	10 %	Escenarios `@bug` y `BUGS.md` con esperado y obtenido

70 %

para aprobar. El enunciado completo se publica en el portal al iniciar el examen.

LO QUE MÁS SUMA

Pocos escenarios **bien** hechos, en verde y con validaciones reales valen más que muchos a medias.

LO QUE MÁS RESTA

Un pipeline en rojo al final. Si un escenario no pasa y no es un bug, quítalo de la regresión con `@wip` y explícalo en el PR.

CANVIA

CIERRE DEL CURSO · PARTE 1

De una frase en español a un pipeline que prueba solo

Martes
HTTP, Gherkin y el primer escenario en verde.

Miércoles
Token, ciclo de vida completo, records y JSON Schema.

Jueves
Código que escala, reportes y caza de bugs.

Viernes
Pipeline, GitHub Pages y examen final.

Siguiente: **Parte 2 · Playwright + TypeScript + Cucumber**

Practica con otra API pública: **reqres.in** · **petstore.swagger.io**